1. Statement of the Problem

1.1 General Description

Student organizations within Babeș-Bolyai University (UBB) organize a wide variety of events throughout the academic year, such as workshops, conferences, training sessions, social activities, competitions, and other activities addressed to students and members of the university community.

The organization of these events involves multiple actors, activities, and information flows. An event may require defining and publishing event information, managing participants and registrations, assigning responsibilities to organizers, communicating with participants, managing venues and schedules, tracking attendance, and producing relevant information for the organization and the university.

The project consists of designing and implementing a **full enterprise architecture application for managing events organized by UBB student organizations**. The application should provide an integrated solution covering the main stages of the event lifecycle, from event creation and planning to participant registration, execution, and post-event activities.

You will work in teams and you are expected to approach the application as an enterprise software system rather than as a standalone prototype. The solution should therefore address the interaction between multiple application layers, persistent data, APIs, user interfaces, business rules, security, testing, and deployment.

The application should be developed using **GenAI as an integral part of the software engineering process**. You are expected not only to use GenAI to generate code, but also to demonstrate how to formulate requirements and prompts, provide context, evaluate generated artifacts, make architectural and implementation decisions, and intervene when the generated solution is incomplete or inadequate.

The project should simulate a realistic software development context in which requirements must be understood, transformed into an architecture and implementation, validated through testing, and ultimately deployed as a working application.

1.2 Overall Requirements

The resulting system must satisfy the following general requirements.

R1. Event Management

The system shall allow authorized users to create, modify, publish, and manage events organized by a student organization.

An event should contain the information necessary to describe and organize it, such as its title, description, date and time, location, organizing organization, capacity, and registration information.

R2. Organization and User Management

The system shall support the management of student organizations and their members.

Different users may have different responsibilities and permissions within the system. The application should therefore provide an appropriate mechanism for authentication, authorization, and access control.

R3. Participant Registration

The system shall allow eligible users to discover available events and register for them.

The application should manage relevant registration information and enforce applicable business rules, such as event capacity or registration status.

R4. Event Lifecycle

The system shall support the relevant stages of an event's lifecycle, from initial creation and planning through publication and registration to the actual event and subsequent activities.

The exact workflows and business rules shall be established by the team based on the requirements-gathering process and subsequently documented in the project specification.

R5. Event Organization

The system shall provide functionality that supports organizers in preparing and managing an event.

Depending on the identified requirements, this may include assigning responsibilities, managing event-related information, monitoring registrations, communicating relevant information, and managing attendance.

The team must justify the functionality included in the final system based on identified user needs rather than introducing functionality solely because it appears technically useful.

R6. Search and Information Access

The system shall provide appropriate mechanisms for users to find and access relevant event information.

Users should be able to identify events that are relevant to them and access the information required to decide whether to participate.

R7. Data Persistence

The application shall use persistent storage for the information managed by the system.

The data model shall be designed to represent the relevant entities, relationships, constraints, and business rules of the application.

R8. Enterprise Architecture

The application shall be designed as a multi-layer enterprise system.

The architecture shall clearly separate relevant concerns, such as presentation, application/business logic, persistence, and external interfaces. Architectural decisions and the technologies selected for implementing them shall be documented and justified.

R9. APIs and Integration

The system shall expose well-defined APIs between its components and, where appropriate, for interaction with external systems.

API design shall be documented and consistent with the architecture of the application.

R10. User Interface

The application shall provide a usable interface for the principal categories of users identified by the team.

The implemented user interface shall correspond to the documented workflows and use cases. The team shall validate important workflows with users and use the results to improve the application.

R11. Testing

The project shall include automated tests appropriate to the implemented system.

In particular, students shall demonstrate that important application behavior is verified through unit tests and, where appropriate, additional levels of testing.

R12. Deployment

The resulting application shall be deployable and runnable outside the individual development environment.

The team shall provide a documented deployment approach and demonstrate the deployed system.

R13. GenAI-Assisted Development

GenAI shall be used throughout the development process as an engineering tool.

Students shall document relevant uses of GenAI, including, where applicable:

- generation and refinement of requirements;
- architectural exploration and decision support;
- generation of code and tests;
- creation or transformation of technical artifacts;

- debugging and problem solving;
- generation and refinement of documentation;
- evaluation and improvement of AI-generated solutions.

The use of GenAI does not replace the students' responsibility for the resulting system. Students must be able to explain, evaluate, modify, test, and defend the artifacts they produce.

R14. Human Oversight and Engineering Judgment

For significant artifacts and functionality, students shall demonstrate a **human-in-the-loop** development process.

The project should make visible how students:

1. provide context to GenAI;
2. formulate and refine prompts;
3. evaluate generated results;
4. identify errors, omissions, or unsuitable design decisions;
5. modify or reject generated solutions;
6. validate the final result through implementation and testing.

The objective is not simply to maximize the amount of AI-generated code, but to demonstrate the ability to use GenAI effectively within a professional software engineering process.

R15. Traceability

The principal requirements, use cases, workflows, architectural decisions, implementation, and tests shall be sufficiently connected so that the team can demonstrate how a requirement is transformed into implemented and validated functionality.

The project should provide evidence that the implemented system corresponds to the requirements identified during the project.

R16. Team-Based Development

The application shall be developed collaboratively by a team of students.

Each student shall have ownership of specific project artifacts and at least one complete cross-layer workflow. Individual responsibilities shall be identifiable in the project documentation and presentations.